

Cardano Oracle — Security Audit

Disclaimer

1. Scope

- Files in scope (on-chain)

- Files reviewed (off-chain, integration context only)

- Out of scope

- Validators / minting policies enumerated

2. Methodology

3. Documentation review

4. Threat model

- Reconciliation against code

5. Findings

- Summary

- H1 — Multiple satisfaction in `existsChangeOutput`

- M1 — Unbounded TEE-signed response datum

- M2 — Permissive `spentTokenNames` check

- M3 — Single-oracle pool UTxO is grief-spendable

- M4 — Off-chain DoS via spam at the request-validator address

- L1 — `unitval` for `SpendingScript` without datum

- L2 — `mkPoolActionID` concatenates without length separators

- L3 — `responseValidityPeriod` not constrained for private pools

- L4 — No `pubkey-length` check in `OracleResponseValidator`

- L5 — `symbols value == [adaSymbol, currencySymbol]` relies on ledger-emitted ordering

- I1 — `unsafeFromBuiltinData` on relayer-supplied redeemer payload

- I2 — Dead code

- I3 — No on-chain TEE attestation

- I4 — Single-oracle pool minting policy lacks one-shot UTxO check

- I5 — No evidence captured from `cabal test`, `cabal outdated`, `pip-audit`

- I6 — Only unit-level CBOR tests; no validator-execution tests

- I7 — Architecture / consumer-integration spec gap

- I8 — No CI test asserting validator-hash byte-equality across rebuilds

- MLabs / Plutonomicon coverage trace

6. Test evidence

7. Remediation status

Cardano Oracle — Security Audit

Disclaimer

This audit was produced by the plutus-auditor AI agent (<https://github.com/quex-tech/plutus-auditor>) fully autonomously, using the `claude-opus-4-7` model, audit skill version `0.1.1`, on `2026-05-15T11:43:34Z`. AI-assisted; not a substitute for human expert review.

Per CIP-52: this audit is point-in-time. It documents what was reviewed and what was found; it does not certify correctness. Subsequent changes to the code or to its off-chain integration are out of scope.

1. Scope

Field	Value
Repository	quex-tech/cardano-oracle (local: /Users/catena/claude_dir/Metawork/projects/quex-tech/cardano-oracle)
Commit SHA	376f9abdf8d551dc2d3ca52f0d09d96b832de9f2
Branch	ai-audit
Working tree	Clean at audit time
Plutus version	V3, plutus-core / plutus-ledger-api / plutus-tx ^>=1.42.0.0
CHaP/Hackage pin	2025-03-05T09:09:31Z (per on-chain/cabal.project)

Files in scope (on-chain)

File	LOC	SHA-256	Role
on-chain/src/Validator.hs	57	93c15ef6... d4051	Shared storageScriptInfo helper
on-chain/src/ OracleResponseValidator.hs	170	bdf9fe75... a78c	Spending + Minting — response UTxOs
on-chain/src/ OracleRequestValidator.hs	198	4404a214... dc046	Spending — request UTxOs
on-chain/src/ SingleOraclePoolValidator.hs	93	d6c74e05... 1287	Spending + Minting — single-oracle pool UTxOs
on-chain/src/ ExampleUserValidator.hs	83	3e1cf4fc... 75e51	Spending — reference consumer (example)
on-chain/test/Main.hs	85	1fc885c7... fa10	Test harness (CBOR + signature self-tests only)

Files reviewed (off-chain, integration context only)

off-chain/{pending_requests.py, responses.py, oracles.py, autorelayer.py, models.py, scripts.py, protocol.py, wallet.py, signer_client.py, http_action.py, utils.py} — reviewed as transaction-builder context for the on-chain validators. Off-chain Python source is not in primary scope.

Out of scope

- quex-v1-signer (TEE/TDX signer enclave) — separate audit.
- TDX hardware / Intel attestation — separate trust root, see threat model.
- Consumer dApp integration patterns — only ExampleUserValidator reviewed as illustration.
- Off-chain Python dependency CVE scan (pip-audit) — environment for installing deps not available during this audit; flagged as Test pending (I5).

Validators / minting policies enumerated

Script	Purpose	Datum	Redeemer
oracleResponseValidatorScript	Spend ∪ Mint	DataItem = (POSIXTimeSeconds, Integer, BuiltinData) (continuing output)	ETHSignedMess = (BuiltinDat ETHSignature) where the payloa (ActionID, DataItem, PubKeyHash)
oracleRequestValidatorScript	Spend	OracleRequest (9-field record incl. reqPoolActionID, reqAfter, reqBefore, reqOwner, reqReward, reqCoinPerUTx0Byte, reqMaxCost)	()
singleOraclePoolValidatorScript	Spend ∪ Mint	(BuiltinByteString, DiffMilliSeconds) (pk, validity period)	same as datum
exampleUserSpendingValidatorScript	Spend	none	()

2. Methodology

Pack	Source	Version
MLabs Plutus Pitfalls (11 categories)	reference/mlabs-vulnerabilities.md	bundled with plutus-audit v0.1.1

Pack	Source	Version
Plutonomicon vulnerabilities (9 categories)	reference/ plutonomicon- vulnerabilities.md	bundled with plutus-audit v0.1.1
CIP-52 audit checklist	reference/cip-52- checklist.md	bundled with plutus-audit v0.1.1

For each validator and minting policy, every category in each pack was walked. Categories not applicable are noted with a one-line justification in §5.

The threat model in `audit/THREAT_MODEL.md` was reused from a prior pass (existed at audit start) and reconciled against the source under audit — see §4.

External tools attempted:

- `cabal test` — **not executed**; Haskell toolchain not available in the audit environment, and the project's Dockerised build (`./compile-contracts.sh`) was not invoked (see Test evidence, §6).
- `cabal outdated`, `pip-audit` — not executed for the same reason. Flagged as Test pending under finding I5.

Off-chain unittest suite under `off-chain/tests/` — **not executed** (Python dependencies not installed in the audit venv; see §6).

3. Documentation review

- `README.md` — present, terse, points to `compile-contracts.sh` and the off-chain `README`. ✓
- `tokens.md` — describes the token set (response, pool, single-oracle pool), lifecycle, and discovery semantics. Cross-checked against on-chain code: pool-action-id derivation (`sha256(cs || tn || actionID)`), datum shape, and asset-class checks match. ✓
- `request-mechanics.md` — describes request datum, spend paths, and the cost formula (`reward + txFee + coinPerUTx0Byte*(274 + responseBytes) - oldResponseCoin`). Matches `OracleRequestValidator.existsChangeOutput`. ✓
- `off-chain/README.md` — present (not re-reviewed here).
- Developer docs referenced in threat model (`tdx/`, `overview/security.md`, `provider/td_oracle_requirements.md`) — external to this repo; assumed accurate per the threat model.

Documentation gap — no architectural state diagram for the response-replacement / consolidation flow, and no public spec of which authority models a consumer-dApp should accept (private pool vs single-oracle pool). Logged as I7.

4. Threat model

The threat model file `audit/THREAT_MODEL.md` was present at audit start and is reused verbatim. SHA-256:
73ea7e8942a150ba3317c603c7542b7471ff971be6226814f9a81a8df754e9cc.

Cardano Oracle: Threat Model

Scope: `on-chain/src/{OracleRequestValidator, OracleResponseValidator, SingleOraclePoolValidator, Validator}.hs` of `quex-tech/cardano-oracle`. Reference: `request-mechanics.md`, `tokens.md`, `off-chain/README.md`, developer-docs (`tdx/`, `overview/security.md`, `provider/td\_oracle\_requirements.md`), `quex-v1-signer` (plutus branch), Halborn EVM audit (`quex-v1-2025-06-25`).

Parties

****Trusted****

- ****TEE / Oracle (Intel TDX enclave running `quex-v1-signer`)****: holds a `secp256k1` signing key generated **inside** the enclave; key never leaves. The corresponding pubkey is bound to the enclave via TDX attestation. Signs `(actionID, dataItem, relayer)` messages. Trust is **verifiable**: TDX hardware + open-source signer source + measurement-attested deployment, not blind operator trust.

****Untrusted****

- ****Relayer****: any Cardano pubkey. Picks up TEE-signed messages and submits fulfillment txs. May drop, delay, reorder, or refuse. Cannot forge: the signature is verified on-chain against the pool pubkey, and the relayer pkh is **inside** the signed payload (MEV protection per `td\_oracle\_requirements.md`).

- ****Requester (`reqOwner`)****: funds and reclaims own request UTx0. Trusted only with own funds (`txSignedBy txInfo reqOwner` gates the expired-reclaim path).

- ****Consumer dApp****: out of scope for these validators; its safety depends on its own checks (see assumption #6).

****Authority (trusted to follow procedure)****

- ****Pool authority****: controls `PoolID` native asset and its movement. For a **private** pool, may add/remove oracles by spending the pool-token UTx0. For a **single-oracle** pool, the pool token is locked at `SingleOraclePoolValidator` with token name `sha256(pubkey || validityPeriod)` and cannot be revoked.

Trust assumptions

1. ****TDX is sound, `quex-v1-signer` has no backdoor, Intel is honest.**** The only blind-trust root. TDX isolates enclave memory from the machine owner; the open-source signer (plutus branch)

does not exfiltrate the key. Both are independently auditable. If either breaks, every signed response is forgeable.

2. ****Pool authority verified attestation before placing the pubkey in the pool datum.**** The on-chain validator does *not* check attestation: it only verifies the signature against whatever pubkey the pool datum contains. The linkage "datum pubkey = TEE-locked private key" lives off-chain at registration time (``verify.quex.tech``).
3. ****Pool authority minted `PoolID` exactly once.**** ``findPoolID`` accepts any single non-ADA asset of quantity 1 in the reference input. It does not check the *minting policy*. Uniqueness is operational, not on-chain enforced. A re-mint silently forks pool authorization.
4. ****TEE clock is monotonic and chain-synced.**** Validators compare TEE-signed timestamps against ``txInfoValidRange``. Skew above ``ResponseValidityPeriod`` makes responses always-stale or perpetually-fresh.
5. ****The audited signer correctly binds `actionId` to the action it executed.**** Per ``td_oracle_requirements.md`` § "responsibility of TD provider". Enforced by signer source: verified once during the ``quex-v1-signer`` audit, not per-response on-chain.
6. ****Consumer dApp enforces response payload schema.**** The response validator only requires the datum to decode as ``(POSIXTime, Integer, BuiltinData)``; the ``BuiltinData`` body is opaque on-chain. Consumers must validate the payload, handle duplicate response UTxOs (allowed per ``tokens.md``), and check freshness themselves.
7. ****`reqCoinPerUTx0Byte` is pinned to live protocol parameters by `pending\_requests.py`.** Requester-supplied datum field; stale values silently break fulfillment economics (DoS to requester, no fund loss).

Properties the protocol provides (verify in Step 3)

- ****Content-addressed `ActionID`**** = ``keccak256(serialized HTTPAction)`` (``quex_backend/models.py:230``). Same request parameters yield the same ``PoolActionID = sha256(PoolID || ActionID)`` and share one response slot. Cached-response reuse is intentional, not a vulnerability.
- ****MEV protection****: relayer pkh is *inside* the TEE-signed payload; validator enforces ``relayer ∈ txInfoSignatories``. A third party cannot re-broadcast a signed response and capture the reward.
- ****Replay protection****: ``responseExpiresAt = signedAt + ResponseValidityPeriod`` checked against ``txInfoValidRange``; replacing an existing response UTxO requires ``newTimestamp > oldTimestamp``.
- ****Untrusted-relayer recourse****: requester recovers funds via the expired-reclaim path (``reqBefore`` lapsed + ``txSignedBy reqOwner``) if no relayer fulfills.

Reconciliation against code

Threat-model claim	Code that enforces / acknowledges it
Relayer is in signed payload; checked via <code>relayer ∈ txInfoSignatories</code>	<code>OracleResponseValidator.hs:93</code> (<code>relayerelem txInfoSignatories</code>) — verified
<code>newTimestamp > oldTimestamp</code> on replacement	<code>OracleResponseValidator.hs:106</code> — verified
Response not expired vs <code>validRange</code>	<code>OracleResponseValidator.hs:145–152</code> (<code>verifyOracleMessage</code>) — verified
Expired-reclaim gated by <code>txSignedBy txInfo owner</code>	<code>OracleRequestValidator.hs:99</code> — verified
Cost cap via <code>reqMaxCost</code>	<code>OracleRequestValidator.hs:129</code> — verified
<code>oldResponseCoin</code> reduces cost	<code>OracleRequestValidator.hs:126–128</code> — verified
<code>findPoolID</code> accepts any single non-ADA asset of qty 1 (assumption #3)	<code>OracleResponseValidator.hs:133–137</code> — verified; this is the trusted hole
Validator does not check TEE attestation (assumption #2)	No on-chain <code>quote_verify</code> anywhere — verified absent

The threat model is consistent with the code. Findings below reference specific assumptions where relevant.

5. Findings

Findings are grouped by severity. Locations are `<file>:<line-range>`. Each finding lists the category from the methodology pack and the test that would catch it.

Summary

ID	Severity	Title
H1	High	Multiple satisfaction in <code>existsChangeOutput</code> — duplicate same-owner same-poolActionID requests share one change output
M1	Medium	Unbounded TEE-signed response datum — DoS on consolidation and consumer-side read
M2	Medium	Permissive <code>spentTokenNames</code> check (<code>_ -> True</code>) accepts script inputs with no policy token

ID	Severity	Title
M3	Medium	Single-oracle pool UTxO is grief-spendable; reference-input revalidation needed by relayers
M4	Medium	Off-chain DoS via spam UTxOs at the request-validator address (no on-chain auth)
L1	Low	Request validator returns <code>unitval</code> (succeeds) for spending inputs without inline datum
L2	Low	<code>mkPoolActionID</code> concatenates <code>cslltnlactionID</code> without length separators
L3	Low	<code>responseValidityPeriod</code> not constrained for private pools (only enforced in <code>SingleOraclePoolValidator</code>)
L4	Low	No <code>pubkey-length</code> check in <code>OracleResponseValidator</code> ; only enforced in <code>SingleOraclePoolValidator</code>
L5	Low	<code>symbols value == [adaSymbol, currencySymbol]</code> relies on ledger-emitted ordering
I1	Informational	<code>unsafeFromBuiltinData</code> on relayer-supplied redeemer payload (CIP-52 §3)
I2	Informational	Dead code: <code>getInlineDatum</code> (response validator), <code>isValidOracleOutput</code> (request validator)
I3	Informational	No on-chain TEE attestation (Plutonomicon-7b) — acknowledged trust assumption
I4	Informational	Single-oracle pool minting policy lacks one-shot UTxO check — by design
I5	Informational	No <code>cabal test / cabal outdated / pip-audit</code> evidence captured during this audit (toolchain absent)
I6	Informational	Only unit-level CBOR tests; no <code>plutus-simple-model</code> or <code>quickcheck-dynamic-validator-execution</code> tests
I7	Informational	Architecture / consumer-integration spec gap (CIP-52 §2)

ID	Severity	Title
18	Informational	No CI test asserting validator-hash byte-equality across rebuilds (Plutonomicon-9)

Counts: 0 Critical / 1 High / 4 Medium / 5 Low / 8 Informational.

H1 — Multiple satisfaction in existsChangeOutput

Severity: High **Location:** on-chain/src/OracleRequestValidator.hs:112–135 **Category:** MLabs-7 (Multiple Satisfaction)

Description. When two request UTxOs with the *same* reqPoolActionID and the *same* reqOwner are spent in one transaction, both invocations of existsChangeOutput validate against the *same set of outputs*. The check is

```
minChange <= 0 || any (\o -> isOwners o && lovelaceValueOf
    (txOutValue o) >= minChange) txOutputs
```

— pure existence (any), with no binding between an input and a specific output. A single change output to reqOwner whose lovelace is $\geq \max(\text{minChange_A}, \text{minChange_B})$ satisfies both validators. The relayer pockets $\min(\text{minChange_A}, \text{minChange_B})$ of ADA that should have been returned.

This is the classical Plutus multiple-satisfaction pattern.

Why not blocked by the response validator’s “exactly one output with cs” rule? That rule restricts the response output, not the change outputs. Two requests for the *same* poolActionID are *intended* to be served by one response UTxO (per tokens.md discovery semantics), so the rule does not gate this case.

Attack vector. 1. Honest user runs ./pending_requests.py add ... twice for the same (pool_id, action) (e.g., dashboard double-click, retry-after-timeout, or distinct duplicate orders). Two request UTxOs at the same reqPoolActionID are created, each locking requestCoin = $\max(\text{min_lovelace_change_utxo} + \text{max_cost}, \dots)$. 2. Relayer bundles both into one fulfillment tx, with one response output and one change output of value minChange to reqOwner. 3. Both request validators pass; relayer collects the extra minChange as wallet change.

Per-attack profit (relayer): one full minChange = requestCoin – cappedCost worth of ADA. With the default CARDANO_FEE_BUFFER = 1_000_000 + RELAYER_REWARD = 50_000 + coins_per_utxo_byte contributions, minChange is on the order of 1.1 – 1.5 ADA per duplicated request. Not catastrophic per event, but scales with duplication frequency.

A second, weaker variant exists across *different* owners with the same reqPoolActionID: each owner gets their own minChange (different isOwners predicate), so this variant only over-counts the txFee and responseBytes cost components, not minChange itself. Still relayer-favourable but not exploitable as “lose owner’s refund”. The first variant (same owner) is the actionable High.

Recommendation. Bind each request input to a *dedicated* change output. Options:

- **(a)** Require the change output to be identified by *index* — add a Output Reference redeemer or store the expected index in the request datum.
- **(b)** Encode the request's TxOutRef (its own UTxO ref) in the *datum* of the change output (e.g., `OutputDatum (Datum (toBuiltinData ownInputRef))`); the validator then requires `findValidChangeOutputFor ownInputRef txOutputs` rather than “any output to owner”.
- **(c)** Reject any tx that consumes more than one request UTxO with the same `reqPoolActionID` (simpler but breaks legitimate batching of identical requests).

1. is the most idiomatic Plutus mitigation.

Test required. `plutus-simple-model`: build a tx with two identical-datum request inputs and one minimal change output; assert validator rejects (currently passes).

Test pending.

M1 — Unbounded TEE-signed response datum

Severity: Medium **Location:** `on-chain/src/`

`OracleResponseValidator.hs:62-110` **Category:** MLabs-4 / Plutonomicon-2

Description. The response datum is `DataItem = (POSIXTimeSeconds, Integer, BuiltinData)`. The third field is opaque `BuiltinData`, signed by the TEE. The on-chain validator imposes no upper bound on its serialised size before writing it to the response UTxO. Subsequent operations that *read* the datum (the response validator on the consolidation path, the request validator's `findValidResponseDatum`, any consumer dApp validator) pay the size cost in execution units and min-utxo lovelace.

Per threat-model assumption #1, the TEE is trusted, so a malicious TEE is out of scope. However, a *buggy* TEE that signs an oversized payload — or an off-chain integration bug that allows arbitrary-size response bodies — would create a permanent DoS for the affected `poolActionID`: every consolidation tx exceeds `maxTxExUnits`, and the response UTxO becomes unsweep-able.

Attack vector. A TEE bug or operator-side misconfiguration produces a multi-kilobyte signed `BuiltinData`. The relayer (acting in good faith) submits the mint tx, which succeeds (the validator's path through `unsafeFromBuiltinData rawDataItem :: DataItem` only extracts `newTimestamp`; the size cost on first write is paid by the relayer). The response UTxO is now stuck; any future “replace with newer response” or “consolidate duplicates” tx must read the bloated datum and fails the ex-unit limit.

Plutonomicon-2 framing: cost shifts to the *next* spender, but here the next spender is the protocol itself — there is no incentive structure to absorb it.

Recommendation. Add a length cap at the validator boundary:

```
let serialisedSize = lengthOfByteString (serialiseData
    rawDataItem)
in serialisedSize <= 4096    -- or some protocol-chosen N
```

```
&& verifyOracleMessage ...
&& ...
```

Choose N based on the realistic schema range. Document the cap in `tokens.md`.

Test required. Build a tx whose `rawDataItem` is `replicateByteString 16384 0x00`-padded; assert validator rejects. **Test pending.**

M2 — Permissive `spentTokenNames` check

Severity: Medium **Location:** `on-chain/src/OracleResponseValidator.hs:101-104`, `on-chain/src/SingleOraclePoolValidator.hs:75-78`, helper at `on-chain/src/Validator.hs:46-57` **Category:** MLabs-8 (Missing UTxO authentication) / informational variant

Description. Both the response validator and the single-oracle-pool validator use:

```
case spentTokenNames of
  [tn] -> tn == poolActionID    -- (or tokenName in the pool
    validator)
  _    -> True
```

The `_ -> True` branch is required to pass the *minting* invocation (where `storageScriptInfo` returns `spentTokenNames = []`). But the same branch also permits *spending* a script-address UTxO that contains **zero** tokens of the own policy, or ≥ 2 tokens of the own policy.

The zero-token case is real: anyone can send a $(2 \text{ ADA}, \emptyset)$ UTxO to the response-validator (or single-oracle-pool) address. When that UTxO is consumed by a future fulfillment / consolidation tx, the validator does not gate it — only the surrounding logic (signed message, output reconstruction) matters. The contributed ADA flows into the tx and ends up as wallet change to the relayer.

Attack vector. A relayer who notices orphan ADA at the script address adds the orphan UTxO as a script input on their next legitimate fulfillment tx. Net cost: zero. Net gain: the orphan's lovelace.

Impact. Loss of *misplaced* ADA only. The mainline protocol does not deposit ADA at the script without a token, so loss is bounded by user/operator mistakes. Severity is Medium rather than Low because the validator advertises itself as a *minting policy address* — operators reasonably expect funds at that address to be locked; without the check they are not.

Recommendation. Branch on `ScriptInfo` directly rather than using a single sentinel:

```
storageScriptInfo :: ScriptInfo -> [TxInInfo] -> StorageScriptInfo
storageScriptInfo (MintingScript cs) _ = ...                -- as
    today, sentinel = []
storageScriptInfo (SpendingScript outRef _) txInfoInputs =
  case findInput outRef txInfoInputs of
    Just (TxOut ... (Value v) ...) ->
```

```

    let tns = maybe [] keys (lookup cs v)
    in (cs, addr, datum, tns)
Nothing -> error ()

```

— then in the validator, require `spentTokenNames == [poolActionID]` on *spending* (no `_` -> `True` fallthrough), and `spentTokenNames == []` on *minting*.

Test required. Place a 5-ADA UTxO with no policy token at the response-validator address; build a legitimate fulfillment tx with that UTxO as an extra script input; assert validator rejects (currently accepts). **Test pending.**

M3 — Single-oracle pool UTxO is grief-spendable

Severity: Medium **Location:** on-chain/src/SingleOraclePoolValidator.hs:59–83 **Category:** MLabs-9 / Plutonomicon-4 (adversarial concurrency)

Description. The single-oracle pool validator permits spending the pool UTxO as long as:

- `oldDatum == rawRedeemer` (the redeemer reproduces the existing datum),
- one continuing output with `(cs, sha256(rawRedeemer), 1)` at the same script address and the same datum.

Anyone — not just the pool authority — can satisfy these constraints, because the datum is part of the redeemer and is public. The spender chooses how much ADA to put in the continuing output (subject to `min_lovelace_post_alonzo`). They keep the excess as wallet change.

In itself this is not a *fund-loss* attack because off-chain `SingleOracleRepository.add_tx` already pins the pool UTxO to `min_lovelace_post_alonzo` (no excess to drain). The *real* impact is grief: the pool UTxO’s `TxOutRef` changes every time anyone spends-and-recreates. Relayer fulfillment txs reference the pool UTxO as a *reference input*; when the referenced UTxO is consumed in a competing block, the relayer’s tx is invalidated and must be rebuilt with the new `TxOutRef`. An adversary can repeatedly cycle the pool UTxO to grief relayer throughput.

Attack vector. Attacker repeatedly submits “no-op” spend-and-recreate txs against the single-oracle pool UTxO at modest fee cost. Each one invalidates pending relayer fulfillment txs that reference it. Each relayer retry costs them ex-units; legitimate request fulfillment latency rises.

Cost asymmetry. Attacker pays only tx fee. Relayer pays tx fee + ex-units re-evaluation. Asymmetry favours the attacker.

Recommendation.

- Require the spending tx to be signed by a *pool authority* pkh embedded in the datum, e.g., extend the datum from `(pubKey, validityPeriod)` to `(pubKey, validityPeriod, authorityPkh)` and check `txSignedBy txInfo authorityPkh` on spend. (Spec change: revocation of a single-oracle pool would then become possible, deviating from the current “cannot be revoked” property — discuss with maintainers before adopting.)

- Or: forbid spending entirely (`SpendingScript ... -> False`) — pool UTxOs become immutable once created, eliminating the grief vector. This is consistent with the stated “cannot be revoked” property.

Option (b) is the cheaper mitigation if the “cannot be revoked” property is intentional.

Test required. `plutus-simple-model`: submit a no-op spend-and-recreate by an arbitrary key; assert rejection. **Test pending.**

M4 — Off-chain DoS via spam at the request-validator address

Severity: Medium **Location:** `on-chain/src/OracleRequestValidator.hs` (entire validator); `off-chain/pending_requests.py:192–200` (`RequestRepository.all`) **Category:** MLabs-10 (Cheap spam)

Description. The request validator does not authenticate request UTxOs by a token — the validator pattern-matches on the datum alone. Anyone can send arbitrary UTxOs (with or without `OracleRequest` datum) to the request-validator address. The autorelayer’s `RequestRepository.all` fetches *all* UTxOs at the address and filters by datum-decode success, paying $O(N)$ per scan.

A spammer can drive N upward at small cost: each spam UTxO requires `min_lovelace_post_alonzo` (a couple of ADA, refundable by the spammer themselves via the `unitval` short-circuit on no-datum UTxOs — see L1). The autorelayer’s scan cost scales linearly.

The recent commit `0a211a0` (“feat: add UTxO blacklist for autorelayer”) is a partial mitigation: operators can blacklist specific spam UTxOs in `config.json`. This is operational, not on-chain.

Recommendation.

- Require each request UTxO to carry an authority NFT minted by a known one-shot policy (e.g., a `Request Authority` token minted on creation), and have the validator check it. This breaks the `pending_requests.py add UX` (extra mint + min-utxo for the NFT) but eliminates the spam vector.
- Or: keep current behaviour and accept the operational mitigation. Document the design choice in `request-mechanics.md` and add rate-limit / cost-cap heuristics in `autorelayer.py`.

Test required. Off-chain: send 100 garbage UTxOs to the request-validator address; assert `RequestRepository.all` remains responsive within an SLA bound. **Test pending.**

L1 — `unitval` for `SpendingScript` without datum

Severity: Low **Location:** `on-chain/src/OracleRequestValidator.hs:189–193` **Category:** MLabs-3 / Plutonomicon-6 (variant)

Description.

```
oracleRequestUntypedValidator responseCurrencySymbol ctx =
  let scriptContext@(ScriptContext _ _ scriptInfo) =
    unsafeFromBuiltinData ctx
  in case scriptInfo of
    SpendingScript _ (Just (Datum datum)) ->
      check (oracleRequestTypedValidator
        responseCurrencySymbol (unsafeFromBuiltinData datum)
        scriptContext)
    SpendingScript _ Nothing -> unitval
  _ -> error ()
```

The `SpendingScript _ Nothing -> unitval` branch returns success for any spending tx whose script input has *no inline datum*. Anyone (including a passer-by) can therefore spend a UTxO that they or someone else accidentally placed at the request-validator address without a datum. The validator imposes no constraints.

This is consistent with a “junk-free” policy (the validator does not lock junk), but it means the address acts as a free-for-all for ADA without inline datum. Combined with M4, it lets a spammer recover their own spam ADA later — making spam cheap.

Recommendation. Replace `unitval` with `error ()`: no-datum UTxOs become unspendable. Cost: a misconfigured user permanently loses any ADA they accidentally send without a datum. Trade-off: prevents free recovery by spammers, but also burns honest mistakes. Document the choice.

Test required. Build a (2 ADA, no datum) UTxO at the request-validator address; attempt to spend with an arbitrary pkh; assert rejection. **Test pending.**

L2 — mkPoolActionID concatenates without length separators

Severity: Low **Location:** on-chain/src/

`OracleResponseValidator.hs:139–142` **Category:** Hardening; not exploitable

Description.

```
mkPoolActionID (AssetClass (CurrencySymbol cs, TokenName tn))
  actionID =
  TokenName . sha2_256 $ cs `appendByteString` tn
    `appendByteString` actionID
```

`cs` is fixed-length (28 bytes, script hash). `tn` is variable (0–32 bytes). `actionID` is fixed (32 bytes, keccak-256 of the serialised HTTP action). Without a length prefix, two distinct triples (`cs`, `tn`, `actionID`) could in principle map to the same concatenation if `tn`’s length varies. In practice:

- `cs` is always 28 bytes.
- `actionID` is always 32 bytes (keccak output).
- An attacker would need to find an HTTP action whose keccak256 output begins with the trailing bytes of a different `tn` — a keccak preimage problem (infeasible).

So this is **not exploitable**. It is a hardening opportunity flagged for code hygiene.

Recommendation. Either fix `tn` to 32 bytes (it already is by convention for sha256-derived names) or use length-prefixed concatenation: `tn_len_byte ++ cs ++ tn ++ actionID`. Document the chosen invariant.

Test required. None — preimage-resistance is assumed.

L3 — `responseValidityPeriod` not constrained for private pools

Severity: Low **Location:** on-chain/src/
`OracleResponseValidator.hs:144–152`; cross-ref
`SingleOraclePoolValidator.hs:66–67` **Category:** Specification gap

Description. `SingleOraclePoolValidator` requires `responseValidityPeriod > 0`. For private pools, the pool UTxO is at an arbitrary off-chain-controlled address — no validator enforces the constraint. If a pool authority creates a private-pool UTxO with `responseValidityPeriod = 0` or negative `DiffMilliSeconds`, every fulfillment using that pool fails (after `responseExpiresAt validRange` always false for non-positive periods).

This is a self-DoS for the pool authority — not a security issue per se — but it is worth flagging because the asymmetry with the single-oracle path is non-obvious.

Recommendation. Mirror the `> 0` check in the response validator's `verifyOracleMessage` (cost: one extra `>` per fulfillment). Or: document the constraint in `tokens.md` so private-pool authorities self-enforce.

Test required. Build a private-pool UTxO with `responseValidityPeriod = 0`; submit a fulfillment; assert rejection with a non-confusing trace. **Test pending.**

L4 — No pubkey-length check in `OracleResponseValidator`

Severity: Low **Location:** on-chain/src/
`OracleResponseValidator.hs:144–159` **Category:** Robustness

Description. `verifyOracleMessage` extracts `pubKey :: ETHCompressedPubKey = BuiltinByteString` from the pool datum and passes it directly to `verifyEcdsaSecp256k1Signature`. The `PlutusTx` builtin returns `False` on a wrong-length pubkey, so the validator does fail — but with a generic “signature invalid” trace, not a clear “wrong pubkey length” trace.

`SingleOraclePoolValidator` does enforce `lengthOfByteString pubKey == 33` (line 66) at *registration*. Pool authorities of single-oracle pools are protected; pool authorities of private pools are not.

Recommendation. Add `lengthOfByteString pubKey == 33` early in `verifyOracleMessage` with a `traceIfFalse` for diagnostic clarity.

Test required. Build a pool UTxO datum with a 32-byte pubkey; submit a fulfillment with any signature; assert rejection with `traceIfFalse "Invalid pubkey length" ...` **Test pending.**

L5 — `symbols value == [adaSymbol, currencySymbol]` relies on ledger-emitted ordering

Severity: Low **Location:** `on-chain/src/OracleResponseValidator.hs:100`, `on-chain/src/SingleOraclePoolValidator.hs:74` **Category:** Robustness

Description. The check uses ordered list equality. In practice, the Cardano ledger emits `Value` keys sorted lex-ascending, with `adaSymbol = CurrencySymbol ""` first. The validator's own `currencySymbol` (a 28-byte script hash) is always lex-greater than the empty bytestring, so the order `[adaSymbol, currencySymbol]` matches.

This is correct *given* the ledger's ordering invariant — but the invariant is not part of the `PlutusLedgerApi` contract advertised to script authors; it's an implementation detail. A future ledger version that reorders `Value` keys (e.g., by canonical CBOR) would silently break this check.

Recommendation. Make the check order-independent. Either:

```
sort (symbols value) == sort [adaSymbol, currencySymbol]
```

(cost: one extra sort) or:

```
length (symbols value) == 2 && adaSymbol `elem` symbols value &&  
    currencySymbol `elem` symbols value
```

Test required. Property test: shuffle the `Value`'s symbol order; assert validator still accepts the same set. **Test pending.**

I1 — `unsafeFromBuiltinData` on relayer-supplied redeemer payload

Severity: Informational **Location:** `on-chain/src/OracleResponseValidator.hs:87–88`, 152 **Category:** CIP-52 §3 (Source-code quality) / MLabs-3 (variant)

Description. The response validator decodes the redeemer's `oracleMessage` via `unsafeFromBuiltinData` at two points. The redeemer is supplied by the relayer (untrusted). If the relayer supplies a malformed CBOR, the validator errors out at run time — wasting their *own* tx submission, not affecting the protocol.

This usage is *permissible* per CIP-52 because:

1. The signature check on the malformed payload would fail anyway (different `serialiseData` output, different keccak hash, different EC recovery).
2. There is no path where successful decode of malformed data leads to a security-relevant action.

Still, CIP-52 §3 asks for “reasonable use of `unsafeFromBuiltinData` — only on data the validator controls”. A defence-in-depth pattern would use `fromBuiltinData` returning `Maybe` and explicit `traceIfFalse` “malformed redeemer” ...

Recommendation. Acceptable as-is. Optional hardening: switch to `fromBuiltinData` on the relayer-supplied outer redeemer; keep `unsafeFromBuiltinData` only for fields the validator has already authenticated (i.e., the post-signature-verify decode of `rawDataItem`).

Test required. None.

I2 — Dead code

Severity: Informational **Location:** on-chain/src/`OracleResponseValidator.hs:113–116` (`getInlineDatum`), on-chain/src/`OracleRequestValidator.hs:151–161` (`isValidOracleOutput`) **Category:** CIP-52 §3 (Source-code quality)

Description. Two helper functions are defined and unused:

- `getInlineDatum :: OutputDatum -> BuiltinData` in `OracleResponseValidator.hs` — no call sites.
- `isValidOracleOutput :: AssetClass -> POSIXTimeSeconds -> POSIXTimeSeconds -> TxOut -> Bool` in `OracleRequestValidator.hs` — logic is duplicated inline in `findValidResponseDatum` (lines 170-184).

Recommendation. Remove unused helpers, or call `isValidOracleOutput` from `findValidResponseDatum` to deduplicate.

Test required. None — would be caught by `hlint/weeder` in CI.

I3 — No on-chain TEE attestation

Severity: Informational (acknowledged trust assumption) **Location:** N/A — absence **Category:** Plutonomicon-7b (Key compromise)

Description. The on-chain code does not verify a TDX attestation quote. The mapping “pool datum’s pubkey ↔ a TEE-locked private key” is a *registration-time* off-chain commitment by the pool authority (per threat-model assumption #2).

If TDX is broken, or if a pool authority registers a non-TEE pubkey, the on-chain code has no defence — signed responses are forged at will.

Recommendation. Acknowledged as design intent. Future improvement path: integrate Intel DCAP quote verification on-chain (cost-prohibitive at current Plutus pricing, but a research direction). Until then, document the assumption prominently in `tokens.md` / consumer-integration docs so that consumer dApps understand they are inheriting the pool authority’s diligence.

Test required. None.

I4 — Single-oracle pool minting policy lacks one-shot UTxO check

Severity: Informational (by design) **Location:** on-chain/src/SingleOraclePoolValidator.hs:58–83 (minting branch) **Category:** Plutonomicon-8 (variant — not exploitable)

Description. The single-oracle pool policy permits anyone to mint a `(cs, sha256(pk||vp), 1)` token by supplying `(pk, vp)` as the redeemer and creating a continuing output at the script address. There is no consumed-UTxO-ref check (the classic one-shot-policy pattern).

This means:

- Multiple identical pool UTxOs for the same `(pk, vp)` can coexist (interchangeable).
- Anyone can self-register their own `(pk_attacker, vp)` and produce a pool UTxO at the validator address.

Why not exploitable. Consumers bind to a specific `poolActionID = sha256(cs_single || sha256(pk_legit || vp) || actionID)`. An attacker-registered pool produces a different `poolActionID` (because `sha256(pk_attacker || vp) ≠ sha256(pk_legit || vp)`), and consumer validators that pin the asset class do not accept it.

Per `tokens.md`: “any token can be a pool token” — anyone-can-self-register is by design.

Recommendation. No change. Add a note in `tokens.md` explicitly explaining the consumer-binding requirement so consumer-dApp authors do not mistakenly bind only to `(cs_single, ?)` patterns.

Test required. None — design.

I5 — No evidence captured from `cabal test`, `cabal outdated`, `pip-audit`

Severity: Informational **Location:** Audit environment limitation **Category:** CIP-52 §8 / Plutonomicon-5

Description. The audit environment lacks `GHC` / `cabal` / `Docker` / a Python `venv` with the off-chain dependencies installed. The audit could not:

- Run `cabal test` (the existing `CBOR/unsafeFromBuiltinData/signature` tests in `on-chain/test/Main.hs`).
- Run `cabal outdated` and cross-reference against `IntersectMBO/plutus` advisories.
- Run `pip-audit` against the off-chain `pyproject.toml`.
- Run the off-chain `unittest` suite (`off-chain/tests/`) — attempted, fails with `ModuleNotFoundError: No module named 'pycardano'` because no `venv` is provisioned.

Recommendation. Re-run this audit in an environment with the Dockerised devx-devcontainer (ghcr.io/input-output-hk/devx-devcontainer:x86_64-linux.ghc96-io) plus uv sync for off-chain, and attach the verbatim outputs as an appendix. This is a *prerequisite* for the audit to be considered CIP-52-compliant.

Test required. N/A — environmental.

I6 — Only unit-level CBOR tests; no validator-execution tests

Severity: Informational **Location:** on-chain/test/Main.hs **Category:** CIP-52 §8

Description. The test suite covers:

- PlutusTx.Builtins.serialiseData against hand-computed hex.
- unsafeFromBuiltinData round-trips for tuples and bytestrings.
- A single verifyEcdsaSecp256k1Signature happy-path test.

It does **not** exercise any validator's mkValidator function end-to-end. There is no plutus-simple-model, cooked-validators, or quickcheck-dynamic harness. Every finding above is marked “Test pending”.

Recommendation. Add a plutus-simple-model harness — minimum coverage:

- Happy-path: request creation → fulfillment → recycle.
- Each High / Medium finding's failure mode as a regression test.

This is the single largest gap relative to CIP-52 §8.

Test required. Build out the harness — out of scope for this audit.

I7 — Architecture / consumer-integration spec gap

Severity: Informational **Location:** docs/ / README.md **Category:** CIP-52 §2 (Documentation review)

Description. No state-machine diagram of the response-replacement / consolidation flow, and no normative spec of which “authority model” a consumer dApp should adopt (must they accept multiple pool models, or only single-oracle? What is the recommended pin?). tokens.md and request-mechanics.md cover individual mechanics well but a consumer-integration guide is absent.

Recommendation. Add docs/consumer-integration.md covering: how to derive the expected poolActionID, how to handle duplicate response UTxOs (per the spec they may exist), and the recommended freshness check.

Test required. N/A.

I8 — No CI test asserting validator-hash byte-equality across rebuilds

Severity: Informational **Location:** N/A (CI absence) **Category:** Plutonomicon-9 (Parameterisation oversights)

Description. The validator hashes (especially the request validator, which is parameterised by the *response validator's* hash) are not pinned in a deployment manifest, nor is there a CI test that rebuilds the validators and asserts their compiled hashes match a checked-in value. A future toolchain upgrade or accidental flag change would silently move the hash and produce a misconfigured deployment.

Recommendation. Add a CI step:

1. Run `./compile-contracts.sh` in a pinned image (already pinned via devcontainer).
2. Assert `sha256(plutus.json) == <pinned-value>`.
3. Tag the manifest in releases.

Test required. Pinned-hash regression test — out of scope for this audit.

MLabs / Plutonomicon coverage trace

For auditability, every category was walked. Categories not raised as findings are documented here with a one-line justification.

Category	Walked	Verdict / location
MLabs-1 (other-redeemer)	✓	Each spend invocation reads its own input via <code>storageScriptInfo's outRef</code> lookup; per-input isolation holds.
MLabs-2 (other-token-name)	✓	Output constraints (<code>currencySymbolValueOf == 1 && symbols value == [ada, cs]</code>) prevent mint smuggling. Note: the validator does not read <code>txInfoMint</code> directly — the constraint is enforced by the single-output-with-cs pattern.
MLabs-3 (arbitrary datum)	✓ → I1	<code>unsafeFromBuiltinData</code> use is bounded by signature verification preceding it.
MLabs-4 (unbounded datum)	✓ → M1	TEE-signed <code>BuiltinData</code> field unbounded.
MLabs-5 (unbounded value)	✓	Continuing outputs enforce <code>symbols value == [ada, cs]</code> . Reference inputs (pool UTXOs) are pool-authority-

Category	Walked	Verdict / location
MLabs-6 (unbounded inputs)	✓	controlled; private-pool junk would self-DoS the authority. storageScriptInfo does one O(N) input lookup per invocation; within tx limits.
MLabs-7 (multiple satisfaction)	✓ → H1	Request validator's existsChangeOutput accepts shared change output across same-owner duplicates.
MLabs-8 (missing UTxO authentication)	✓ → M2	Permissive _ → True lets script inputs with zero policy tokens pass.
MLabs-9 (UTxO contention)	✓ → M3	Single-oracle pool UTxO grief-spendable.
MLabs-10 (cheap spam)	✓ → M4	Request-validator address has no on-chain auth.
MLabs-11 (insufficient staking-key control)	✓	All script addresses constructed with no stake credential (Address payment_part network=nw). No rewards earned; nothing to leak.
P-1 (token-dust spam)	✓	Bounded by output constraints (cf. MLabs-5).
P-2 (large-datum)	✓ → M1	Same root cause as MLabs-4.
P-3 (lack of staking control)	✓	Same as MLabs-11.
P-4 (eUTXO concurrency DoS)	✓ → M3	Pool-UTxO grief.
P-5 (PAB DoS / aeson CVE)	✓ → I5	Off-chain Python parses JSON via requests.response.json(); not audited for CVEs in this pass (toolchain absent).
P-6 (unauth data modification)	✓	Datum equality on response (newDatum == rawDataItem) and single-oracle pool (oldDatum == rawRedeemer) is total — no partial-update bypass. Request validator destroys the request UTxO; nothing to preserve.
P-7a (DNS/data-source)	✓ → I3	Out-of-band; TEE-side, not on-chain.
P-7b (key compromise)	✓ → I3	TEE-locked key; no on-chain attestation.
P-7c (price manipulation)	✓	Consumer responsibility (threat-model assumption #6).

Category	Walked	Verdict / location
P-8 (infinite mint)	✓ → I4	Single-oracle pool policy lacks one-shot check, but consumer pin prevents abuse. Response policy is gated by signature + single-output constraint.
P-9 (parameterisation)	✓ → I8	No CI hash-pin test.
CIP-52 §1 Scope	✓	Captured in §1.
CIP-52 §2 Documentation review	✓ → I7	Gap.
CIP-52 §3 Source-code quality	✓ → I1, I2	unsafeFromBuiltinData use, dead code. No error/undefined in on-chain logic beyond the error () defensive failures (acceptable).
CIP-52 §4 Specification compliance	✓	tokens.md and request-mechanics.md traced to lines (§3 reconciliation table above).
CIP-52 §5 Transaction format	✓	Off-chain responses.py / pending_requests.py build txs consistent with the on-chain shape; reference inputs vs spent inputs distinction is correct.
CIP-52 §6 Vulnerability assessment	✓	This whole §5.
CIP-52 §7 Off-chain code review	✓	Reviewed; flagged M4 (spam) and L1 (unitval).
CIP-52 §8 Test coverage	✓ → I6	Severe gap.
CIP-52 §9 Build reproducibility	✓ → I8	Devx-devcontainer image pinned; hash CI assertion missing.
CIP-52 §10–12 Findings format / severity / disclaimer	✓	Applied throughout.

6. Test evidence

Test	Status	Notes
cabal test (existing CBOR + signature suite, on-chain/test/Main.hs)	⚠ Not run — Haskell toolchain absent in audit environment (which cabal → not found). The suite covers	

Test	Status	Notes
		serialiseData cases, unsafeFromBuiltinData round-trips, and one secp256k1 happy-path. None of the audit's findings have on-chain regression tests in this suite.
./compile-contracts.sh (Dockerised rebuild → blueprints)	⚠ Not run — Docker not invoked in audit environment.	
off-chain/tests/ (unittest)	⚠ Not run — python3 -m unittest discover tests fails with ModuleNotFoundError: No module named 'pycardano' because no venv with the pinned pyproject.toml dependencies was provisioned during the audit.	
cabal outdated against IntersectMBO/plutus advisories	⚠ Not run — cabal absent. CHaP/Hackage index pinned to 2025-03-05 (cf. cabal.project); plutus-core/plutus-ledger-api/plutus-tx all ^>=1.42.0.0. Manual cross-check against the IntersectMBO/plutus advisory feed not performed.	
pip-audit against off-chain/pyproject.toml	⚠ Not run — toolchain absent. Spot-check of pinned versions: pycardano==0.17.0, cryptography==46.0.3, requests==2.32.5, eth-keys==0.7.0 — all 2025-current, no obvious red flags, but no formal CVE scan.	

Outstanding regression tests (per Step 4 of the skill workflow):

- H1: dual same-owner same-poolActionID requests → relayer multiple-satisfaction → asserted-fail
- M1: 16-KiB TEE-signed datum → asserted-fail (or asserted-ex-units-cap)

- M2: orphan (ADA, $\emptyset$) UTxO at script address $\rightarrow$ asserted-fail when used as script input
- M3: arbitrary-key spend-and-recreate of single-oracle pool UTxO $\rightarrow$ asserted-fail
- M4: 100-spam-UTxO scan benchmark for `autorelayer.py`
- L1: (ADA, no datum) UTxO at request-validator address $\rightarrow$ asserted-fail
- L3: `responseValidityPeriod = 0` private pool $\rightarrow$ asserted-fail with informative trace
- L4: 32-byte pubkey in pool datum $\rightarrow$ asserted-fail with informative trace
- L5: shuffled Value symbol order $\rightarrow$ asserted-pass (robustness)
- I8: validator-hash byte equality across rebuilds $\rightarrow$ CI regression

All are **Test pending** — they should be implemented under `plutus-simple-model` and gated by `cabal test` before the next release candidate.

7. Remediation status

ID	Status	Notes
H1	Open	Recommend datum-binding option (b). Awaiting maintainer decision.
M1	Open	Choose N; document in <code>tokens.md</code> .
M2	Open	Refactor <code>storageScriptInfo</code> to branch on <code>ScriptInfo</code> .
M3	Open	Decide spec: hard-immutable pool UTxO (preferred) vs authority-gated spend.
M4	Open	Authority NFT (large change) vs accept (document).
L1	Open	Replace <code>unitval</code> with <code>error ()</code> if junk-loss is acceptable.
L2	Open	Length-prefix or fix <code>tn</code> to 32 bytes.
L3	Open	Mirror <code>> 0</code> check in response validator.
L4	Open	Add <code>lengthOfByteString pubkey == 33</code> with <code>traceIfFalse</code> .
L5	Open	Order-independent comparison.
I1	Open	

ID	Status	Notes
		Optional defence-in-depth.
I2	Open	Delete or use.
I3	Acknowledged	Acknowledged in threat-model assumption #1-#2.
I4	Acknowledged	By-design per <code>tokens.md</code> .
I5	Open	Re-run audit in toolchain-equipped environment.
I6	Open	Build <code>plutus-simple-model</code> harness.
I7	Open	Author docs/ <code>consumer-integration.md</code> .
I8	Open	Add hash-pin CI step.

End of report.